1º DAM / DAW · SISTEMAS INFORMÁTICOS · UD3: GESTIÓN DE LA INFORMACIÓN

Boletín de ejercicios · Parte 1: terminal, árbol y shell

Parte A · Preguntas de reflexión — Parte B · Ejercicios prácticos — Parte C · Test de teoría

Alumno/a: Entrega: Alexia Classroom

Cómo trabajar este boletín: los ejercicios se hacen en tu VM Ubuntu. En las preguntas de reflexión se valora el *porqué*, no la longitud. En los prácticos, deja constancia de los comandos que ejecutas: vale más un error bien diagnosticado que un resultado sin explicación.

Parte A · Preguntas de reflexión

8 preguntas · razona en 3–6 líneas

A1 **Un compañero acaba de crear un fichero, ejecuta `locate` y el sistema le dice que no existe. Sin embargo, el fichero está ahí. ¿Qué ha pasado y cómo lo arregla?**

A2 **¿Por qué se dice que en Linux «todo es un fichero»? Pon dos ejemplos del árbol que lo demuestren y explica qué ventaja aporta esa idea.**

A3 **¿Por qué `vi` se considera imprescindible en un administrador si `nano` es mucho más fácil? ¿Bajo qué circunstancia real agradecerás saber salir de `vi`?**

A4 Borras `original.txt` y el enlace duro sigue funcionando, pero el enlace simbólico queda roto. Explica la diferencia usando la palabra *inodo*.

A5 ¿Qué diferencia hay entre `cmd1 ; cmd2` y `cmd1 && cmd2`? Describe una situación real donde usar `&&` te evite un desastre.

A6 Ejecutas un comando y justo después `echo $?` muestra `0`. ¿Qué significa? ¿Y qué tipo de número verías si el comando hubiera fallado?

A7 Windows centraliza la configuración en el registro; Linux la reparte en ficheros de texto bajo `/etc`. Ventaja e inconveniente de cada enfoque.

A8 Explica con tus palabras por qué `ls *.txt` funciona *aunque* `ls` no sepa nada de asteriscos.

Parte B · Ejercicios prácticos

en tu VM Ubuntu · deja comandos y resultado

B1 Supervivencia. Averigua, sin salir de la terminal: tu usuario, la versión exacta de Ubuntu y la versión del kernel.

COMANDOS Y RESULTADO

B2 Casa nueva. Con una sola orden, crea la estructura `proyectos/2026/informes` y `proyectos/2026/facturas` dentro de tu `/home`. Después crea `nota.txt` dentro de `informes`, cópiala a `facturas` como `copia.txt`, renómbrala a `final.txt` y borra `nota.txt`. Indica qué orden borraría `proyectos` entero y por qué no lo haces.

COMANDOS Y RESULTADO

B3 Caza FHS. Localiza con comandos (`find` o `locate`): a) el fichero de configuración de red de tu VM (pista: `/etc/netplan`); b) los tres ficheros más grandes de `/var/log`; c) todos los ficheros modificados hoy en tu directorio personal.

COMANDOS Y RESULTADO

B4 find contra locate. Crea `pruebalocate.txt` en tu escritorio. Comprueba con `locate` (¿aparece?). Ejecuta lo necesario para que `locate` lo encuentre. Apunta la secuencia completa.

COMANDOS Y RESULTADO

B5 Filtros y tuberías. Con una sola línea de comandos cada vez: a) lista solo los nombres de usuario de `/etc/passwd`, ordenados alfabéticamente; b) cuenta cuántos usuarios hay en el sistema (con una tubería, sin contar a mano); c) muestra solo la primera columna del resultado anterior.

COMANDOS Y RESULTADO

B6 Enlaces. Crea `documento.txt` con cualquier texto. Crea un enlace duro `duro.txt` y un enlace simbólico `soft.txt`. Ejecuta `ls -li` y explica qué ves. Ahora borra `documento.txt`: ¿qué pasa con cada enlace? Compruébalo con `cat`.

COMANDOS Y RESULTADO

B7 Globs. Crea en una carpeta de prueba `file1.txt file2.txt file3.txt nota1.txt nota2.txt`. Predice ANTES de ejecutar qué listará cada comando y compruébalo: a) `ls f*.txt` · b) `ls file?.txt` · c) `ls [fn]*.txt`

PREDICCIÓN Y COMPROBACIÓN

B8 Redirecciones. a) Guarda la lista de tu directorio personal en `lista.txt` y añade después el contenido de `/etc/hostname` al mismo fichero sin sobrescribir. b) Ejecuta un comando que genere un error de permisos (pista: intenta listar `/root` sin `sudo`) y guarda **solo el error** en `errores.txt`. c) Repite enviando el error al «agujero negro».

COMANDOS Y RESULTADO

B9 Tu primer script. Crea `saludo.sh` que: muestre «Buenos días, soy TUUSUARIO», la fecha de hoy (con sustitución de comandos) y el número de ficheros de tu directorio personal (con una tubería). Dale permisos de ejecución y lánzalo con `./saludo.sh`. Pega el contenido del script.

SCRIPT, COMANDOS Y RESULTADO

B10 Misión integradora. Encadena todo: crea una carpeta `~/boletin-ud3` con un enlace simbólico en tu escritorio que apunte a ella; dentro, un script `informe.sh` que redirija a `informe.txt` la fecha, tu usuario y el espacio libre del disco (`df -h`); ejecútalo y verifica con `cat` que el informe existe y es correcto.

COMANDOS Y RESULTADO

Parte C · Test de teoría

12 preguntas · una sola opción correcta · sin penalización

C1 ¿Qué comando muestra el manual completo de otro comando?

a) `ls --help`
b) `man`
c) `info -a`
d) `doc`

C2 Para salir de `vi` descartando los cambios:

a) `:w`
b) `Ctrl+X`
c) `:q!`
d) `exit`

C3 El directorio donde Linux guarda la configuración del sistema:

a) `/var`
b) `/home`
c) `/etc`
d) `/usr`

C4 Los registros (logs) que crecen con el uso del sistema viven en:

a) `/tmp`
b) `/var/log`
c) `/bin`
d) `/proc`

C5 `docs/notas.txt` es una ruta:

a) absoluta
b) relativa
c) simbólica
d) virtual

C6 ¿Qué comando responde exactamente dónde está el binario que estás ejecutando?

a) `whereis` o `which`
b) `locate`
c) `find / -name`
d) `du -sh`

C7 `find . -mtime -1` **busca ficheros:**

- **a)** de más de 1 MB
- **b)** modificados en las últimas 24 horas
- **c)** llamados `-1`
- **d)** de hace más de un día

C8 **El enlace que queda roto si borras el original es:**

- **a)** el duro
- **b)** el simbólico
- **c)** ambos
- **d)** ninguno

C9 **Cuando escribes** `ls *.txt`**, quién interpreta el asterisco:**

- **a)** `ls`
- **b)** la shell, antes de lanzar `ls`
- **c)** el kernel
- **d)** el sistema de archivos

C10 `mkdir datos && cd datos` **ejecuta** `cd` **solo si:**

- **a)** siempre
- **b)** `mkdir` acabó con código 0
- **c)** `mkdir` falló
- **d)** el directorio ya existía

C11 `comando 2> errores.txt` **redirige:**

- **a)** la salida normal
- **b)** la entrada
- **c)** los errores
- **d)** todo a la vez

C12 **La primera línea** `#!/bin/bash` **de un script sirve para:**

- **a)** comentar el fichero
- **b)** indicar el intérprete que lo ejecuta
- **c)** darle permisos
- **d)** nada, es decorativa